Technical Specification: Local Vision-Language Image Tagging Service

Target Architecture:	Cross-Platform Desktop Application (.NET 10 / Avalonia UI)
Component Legacy:	In-Process Local Machine Learning Model Inference Engine
Core Dependencies:	LLamaSharp, LLamaSharp.Backend.Cuda12 (or CPU fallback)
Target Model:	Liquid Foundation Model (Liquid AI LFM2.5-VL-1.6B GGUF)

1. Executive Summary & Design Goals

This document details the software design for integrating local, in-process multi-modal vision-language capabilities into a .NET 10 desktop application built using the Avalonia UI framework. To eliminate external network dependencies, privacy vulnerabilities, and API usage fees, all computational inference tasks must execute locally on the client machine.

The architectural design encapsulates `LLamaSharp` to load and execute quantized GGUF format weights of Liquid AI's **LFM2.5-VL-1.6B** model. This implementation isolates execution from the user interface thread, provides real-time asynchronous token streaming via `IAsyncEnumerable<string>`, implements reactive progress reporting, and establishes clean state management designed to plug directly into an MVVM-compliant architecture.

2. System Dependencies & Environment Setup

The implementation relies on native wrappers communicating via P/Invoke to compiled `llama.cpp` runtimes. The following NuGet package references must be added to the project file:

Package Name	Target Platform / Feature	Purpose
`LLamaSharp`	Core Library (All Platforms)	Provides .NET abstractions, context management, and high-level wrappers for execution.
`LLamaSharp.Backend.Cuda12`	Windows / Linux (NVIDIA GPU)	Native runtime supporting hardware acceleration via CUDA 12 execution blocks.
`LLamaSharp.Backend.Cpu`	Cross-Platform Fallback	Software fallback execution layers for machines lacking compliant graphics cards.

> **Operational Warning for Vibe Coding Engines**

> Ensure that native backends do not collide. If multi-platform targeting is required, configure hardware runtimes conditionally within the `.csproj` workspace or provide clean runtime dependency separation during packaging.

3. Architecture Blueprint & Domain Contracts

To preserve alignment with high-performance desktop development paradigms, all operations are governed by highly specialized abstractions. The processing subsystem consists of three discrete operational components: initialization configuration vectors, execution engine states, and the concrete service delivery wrapper.

3.1 Service Contracts

```
public record ImageTaggingConfig(
    string ModelPath,
    string MultiModalProjectorPath,
    int ContextWindowSize = 2048,
    int GpuLayerCount = 99,
    float Temperature = 0.1f);

public interface IImageTaggingService : IDisposable
{
    bool IsModelLoaded { get; }
    Task InitializeAsync(ImageTaggingConfig config, CancellationToken cancellationToken =
default);
    IAsyncEnumerable<string> GenerateTagsStreamAsync(byte[] imageBytes, string prompt,
CancellationToken cancellationToken = default);
}
```

4. Concrete Engine Implementation

The concrete implementation strictly manages unmanaged memory cycles exposed through underlying C++ bindings. Disposal patterns must guarantee the ordered release of context frames, model weights, and multi-modal projection weights.

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Runtime.CompilerServices;
using System.Threading;
using System.Threading.Tasks;
using LLama;
using LLama.Common;

namespace Adam.Core.Services.ImageTagging
{
    public class LocalImageTaggingService : IImageTaggingService
    {
        private LLamaWeights? _weights;
        private LLamaClipEmbedder? _clipEmbedder;
        private ImageTaggingConfig? _config;
        private readonly SemaphoreSlim _lock = new(1, 1);
        private bool _isDisposed;

        public bool IsModelLoaded => _weights != null && _clipEmbedder != null;

        public async Task InitializeAsync(ImageTaggingConfig config, CancellationToken
cancellationToken = default)
        {
            await _lock.WaitAsync(cancellationToken);
            try
            {
                if (IsModelLoaded && _config?.ModelPath == config.ModelPath)
                    return;

                DisposeLoadedModels();

                _config = config;

                if (!File.Exists(_config.ModelPath))
                    throw new FileNotFoundException("Base GGUF model binary could not be
found", _config.ModelPath);
                if (!File.Exists(_config.MultiModalProjectorPath))
                    throw new FileNotFoundException("Multi-modal projection binary could
not be found", _config.MultiModalProjectorPath);
```

```
                var parameters = new ModelParams(_config.ModelPath)
                {
                    ContextSize = (uint)_config.ContextWindowSize,
                    GpuLayerCount = _config.GpuLayerCount,
                    Seed = 42
                };

                // Offload initialization of heavy model blocks safely to the thread pool
                _weights = await Task.Run(() => LLamaWeights.LoadFromFile(parameters),
cancellationToken);
                _clipEmbedder = await Task.Run(() =>
LLamaClipEmbedder.Create(_config.MultiModalProjectorPath), cancellationToken);
            }
            finally
            {
                _lock.Release();
            }
        }

        public async IAsyncEnumerable<string> GenerateTagsStreamAsync(
            byte[] imageBytes,
            string prompt,
            [EnumeratorCancellation] CancellationToken cancellationToken = default)
        {
            if (!IsModelLoaded || _weights == null || _clipEmbedder == null || _config ==
null)
            {
                throw new InvalidOperationException("The Image Tagging Engine has not been
initialized or weights are missing.");
            }

            if (imageBytes == null || imageBytes.Length == 0)
            {
                throw new ArgumentException("Provided target payload contains empty image
metrics.", nameof(imageBytes));
            }

            await _lock.WaitAsync(cancellationToken);

            LLamaContext? context = null;
            try
            {
                var parameters = new ModelParams(_config.ModelPath)
                {
                    ContextSize = (uint)_config.ContextWindowSize,
                    GpuLayerCount = _config.GpuLayerCount
                };

                context = _weights.CreateContext(parameters);
```

```
                // Process and encode image using the vision model projector
                var imageEmbedding = await Task.Run(() =>
_clipEmbedder.CreateImageEmbedding(imageBytes), cancellationToken);

                var executor = new StatefulExecutor(context);
                var history = new ChatHistory();
                history.AddMessage(AuthorRole.User, prompt, imageEmbedding);

                var inferenceParams = new InferenceParams
                {
                    Temperature = _config.Temperature,
                    AntiPrompts = new[] { "User:", "<|im_end|>" },
                    MaxTokens = 256
                };

                await foreach (var token in executor.InferAsync(history, inferenceParams,
cancellationToken).WithCancellation(cancellationToken))
                {
                    yield return token;
                }
            }
            finally
            {
                context?.Dispose();
                _lock.Release();
            }
        }

        private void DisposeLoadedModels()
        {
            _clipEmbedder?.Dispose();
            _clipEmbedder = null;

            _weights?.Dispose();
            _weights = null;
        }

        public void Dispose()
        {
            if (_isDisposed) return;
            DisposeLoadedModels();
            _lock.Dispose();
            _isDisposed = true;
            GC.SuppressFinalize(this);
        }
    }
}
```

5. Presentation Layer & Avalonia MVVM Bindings

The interface design decouples computing bottlenecks from main rendering operations. Data changes are communicated via Observable features, and streaming tokens are systematically accumulated to avoid rendering lag or stuttering.

5.1 Reactive ViewModel Integration

```
using System;
using System.Collections.ObjectModel;
using System.IO;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

namespace Adam.Modules.Tagging.ViewModels
{
    public partial class ImageTaggingViewModel : ObservableObject
    {
        private readonly IImageTaggingService _taggingService;
        private CancellationTokenSource? _cts;

        [ObservableProperty] private string _imagePath = string.Empty;
        [ObservableProperty] private string _executionOutput = string.Empty;
        [ObservableProperty] private bool _isProcessing;
        [ObservableProperty] private string _statusText = "Idle";

        public ObservableCollection<string> ParsedTags { get; } = new();

        public ImageTaggingViewModel(IImageTaggingService taggingService)
        {
            _taggingService = taggingService;
        }

        [RelayCommand]
        private async Task ProcessImageAsync()
        {
            if (!File.Exists(ImagePath))
            {
                StatusText = "Error: Targeted asset missing on drive.";
                return;
            }

            IsProcessing = true;
            ExecutionOutput = string.Empty;
```

```
            ParsedTags.Clear();
            StatusText = "Initializing Pipeline Neural Weights...";

            _cts = new CancellationTokenSource();

            try
            {
                // Ensure engine baseline is functional
                var config = new ImageTaggingConfig(
                    ModelPath: "models/LFM2.5-VL-1.6B-Q4_0.gguf",
                    MultiModalProjectorPath: "models/mmproj-LFM2.5-VL-1.6B-Q4_0.gguf"
                );

                await _taggingService.InitializeAsync(config, _cts.Token);

                StatusText = "Running Image Vector Projections & Tag Extraction...";
                byte[] rawPayload = await File.ReadAllBytesAsync(ImagePath, _cts.Token);

                string dynamicPrompt = "Identify the distinct subjects, contextual styles,
and explicit items visible. Return as a clean comma-separated list.";

                var stream = _taggingService.GenerateTagsStreamAsync(rawPayload,
dynamicPrompt, _cts.Token);
                var stringBuilder = new StringBuilder();

                await foreach (var token in stream)
                {
                    stringBuilder.Append(token);
                    ExecutionOutput = stringBuilder.ToString();
                }

                ParseOutputToTagsCollection(ExecutionOutput);
                StatusText = "Inference sequence processed cleanly.";
            }
            catch (OperationCanceledException)
            {
                StatusText = "Operation cancelled by user interface interrupt.";
            }
            catch (Exception ex)
            {
                StatusText = $"Fatal Fault Encountered: {ex.Message}";
            }
            finally
            {
                IsProcessing = false;
                _cts?.Dispose();
                _cts = null;
            }
        }
```

```
        [RelayCommand]
        private void CancelInference() => _cts?.Cancel();

        private void ParseOutputToTagsCollection(string cleanText)
        {
            ParsedTags.Clear();
            var extractedSegments = cleanText.Split(new[] { ',', ';', '.' },
StringSplitOptions.RemoveEmptyEntries);
            foreach (var segment in extractedSegments)
            {
                string tag = segment.Trim();
                if (!string.IsNullOrEmpty(tag) && !ParsedTags.Contains(tag))
                {
                    ParsedTags.Add(tag);
                }
            }
        }
    }
}
```

5.2 Declarative View Bindings (Avalonia XAML)

The UI design ensures smooth interactions during heavy background execution. Use text streaming layouts along with explicit layout containment behaviors to protect structural layout positioning from breaking.

```
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2000/xaml"
             xmlns:vm="clr-namespace:Adam.Modules.Tagging.ViewModels"
             x:Class="Adam.Modules.Tagging.Views.ImageTaggingView"
             x:DataType="vm:ImageTaggingViewModel">

    <Grid RowDefinitions="Auto, *, Auto" Margin="20">

        <!-- Module Control Ribbon -->
        <StackPanel Grid.Row="0" Orientation="Vertical" Spacing="10" Margin="0,0,0,15">
            <TextBlock Text="Local Computer Vision Automated Tagging Console"
                       FontSize="16" FontWeight="SemiBold" Foreground="#1a365d"/>
            <Grid ColumnDefinitions="*, Auto, Auto">
                <TextBox Grid.Column="0" Text="{Binding ImagePath}" Watermark="Absolute
Path to Image Asset..." VerticalContentAlignment="Center"/>
                <Button Grid.Column="1" Content="Analyze Asset" Command="{Binding
ProcessImageAsyncCommand}" IsEnabled="{Binding !IsProcessing}" Margin="8,0,0,0"
HotKey="Enter"/>
                <Button Grid.Column="2" Content="Cancel" Command="{Binding
CancelInferenceCommand}" IsEnabled="{Binding IsProcessing}" Margin="5,0,0,0"
Background="#e53e3e" Foreground="White"/>
            </Grid>
        </StackPanel>

        <!-- Central Dynamic Content Frame -->
        <Grid Grid.Row="1" ColumnDefinitions="*, *" Margin="0,10">
            <!-- Left Column: Visual Asset Render -->
            <Border Grid.Column="0" Background="#f7fafc" BorderBrush="#e2e8f0"
BorderThickness="1" CornerRadius="4" Margin="0,0,10,0">
                <Image Source="{Binding ImagePath, Converter={StaticResource
StringToBitmapConverter}}" Margin="10" Stretch="Uniform">
                    <Image.ToolTip>
                        <ToolTip Content="Target image to pass into local neural vision-
language model engine pipeline" />
                    </Image.ToolTip>
                </Image>
            </Border>

            <!-- Right Column: Generative Inference Diagnostics -->
            <Grid Grid.Column="1" RowDefinitions="*, 120" Margin="10,0,0,0">
                <!-- Parsed Tag Micro-Blocks -->
                <Border Grid.Row="0" BorderBrush="#cbd5e0" BorderThickness="1"
CornerRadius="4" Padding="10" Background="#ffffff">
                    <ScrollViewer VerticalScrollBarVisibility="Auto">
```

```
                    <ItemsControl ItemsSource="{Binding ParsedTags}">
                        <ItemsControl.ItemsPanel>
                            <ItemsPanelTemplate>
                                <WrapPanel Orientation="Horizontal" />
```

6. Error Management & Execution Constraints

- **Memory Safety Constraints:** Since `LLamaContext` allocates persistent heap blocks within unmanaged memory pools, instances must be strictly scoped with using blocks or explicit cleanup loops. Failing to free context frames will result in progressive memory leaks across tagging operations.

- **Hardware Execution Policies:** The engine shifts execution onto background workers via `Task.Run`. Native C++ layers handle threading internally based on core availability. Ensure that the system does not invoke parallel processing pipelines over a single active context instantiation.

- **UI Responsiveness:** Text and collections are bound directly to UI threads. The token loop ensures thread-safe dispatching so that background processing does not stall frame updates in Avalonia's rendering pipeline.